

COMPILADORES

Aula 9 — Análise Sintática Descendente

Recursão à esquerda • Fatoração à esquerda • Conjuntos FIRST e FOLLOW

Curso de Ciência da Computação — 2026/2
Projeto integrador: compilador da linguagem MINIC

1. Objetivos de aprendizagem

Ao final desta aula, o estudante deverá ser capaz de preparar uma gramática para análise descendente e justificar as decisões de expansão de um parser preditivo.

- Explicar o princípio da análise top-down e da descida recursiva.
- Identificar recursão à esquerda direta e indireta.
- Eliminar recursão à esquerda preservando a linguagem gerada.
- Aplicar fatoração à esquerda para adiar decisões de produção.
- Calcular FIRST e FOLLOW e interpretar seu papel no reconhecimento preditivo.
- Relacionar as transformações à gramática e ao parser da MINIC.

Conexão com o plano

A Aula 9 corresponde ao Capítulo 4, Seção 4.4, do livro-texto. A Aula 8 estabeleceu a estrutura dos programas e a Aula 10 avançará para LL(1), tabelas preditivas e recuperação de erros.

2. Visão geral: parsing top-down

Na análise descendente, o parser começa pelo símbolo inicial e tenta construir uma derivação mais à esquerda da cadeia de entrada. A árvore é expandida da raiz para as folhas, acompanhando o próximo token disponível.

Ideia central: escolher uma produção para um não-terminal, reconhecer seus símbolos e continuar até consumir a entrada.

O método de descida recursiva associa, em geral, um procedimento a cada não-terminal. Quando o procedimento A é ativado, ele reconhece uma das produções de A.

```
void A() { escolhe  $A \rightarrow X_1 X_2 \dots X_k$ ; reconhece  $X_1$ ; ...; reconhece  $X_k$ ; }
```

A forma geral pode exigir retrocesso: se uma alternativa falhar, o parser retorna ao ponto de escolha e tenta outra. Para linguagens de programação, o objetivo é evitar esse custo por meio de gramáticas adequadas e decisões preditivas.

Figura de apoio do livro

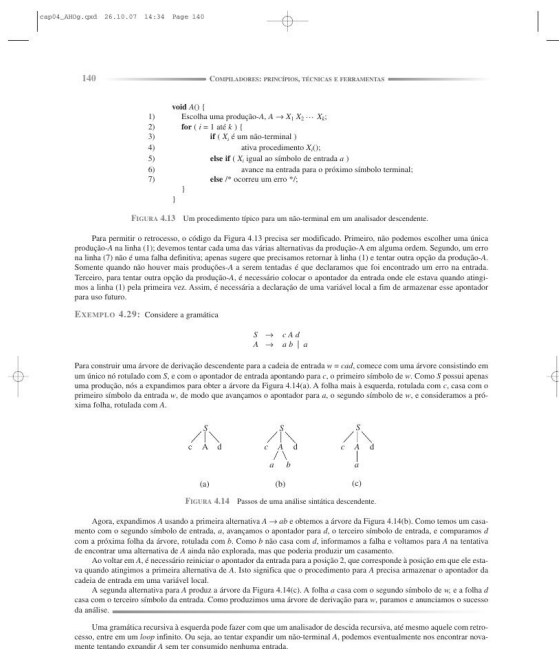


Figura 4.13 e exemplo associado à Figura 4.14 — procedimento típico e passos de uma análise descendente. Fonte: Aho, Sethi e Ullman, Cap. 4, p. 140.

3. Recursão à esquerda

Uma gramática possui recursão à esquerda quando um não-terminal pode derivar uma cadeia cujo primeiro símbolo é ele próprio. Na forma direta, isso aparece em produções como $A \rightarrow A\alpha \mid \beta$.

Por que isso é um problema? Um parser de descida recursiva que expanda A chamando novamente A pode entrar em recursão infinita sem consumir nenhum token.

Exemplo problemático

```
E → E + T | T
T → T * F | F
F → ( E ) | id
```

A transformação clássica para eliminar a recursão direta é:

```
A → Aα | β ■ A → βA'
A' → αA' | ε
```

Aplicação às expressões

```
E → TE'
E' → +TE' | ε
T → FT'
T' → *FT' | ε
```

A linguagem gerada é preservada, mas a estrutura passa a ser adequada para expansão orientada pelo próximo token. Em uma implementação, E' e T' podem ser representados por laços que acumulam operadores e operandos na AST.

4. Recursão indireta e transformação sistemática

A recursão também pode ocorrer por uma cadeia de não-terminais. Por exemplo, $A \Rightarrow B\gamma$ e $B \Rightarrow A\delta$ produz recursão à esquerda indireta. Nesse caso, eliminar apenas produções da forma $A \rightarrow A\alpha$ não é suficiente.

Procedimento de alto nível

- Ordenar os não-terminais $A_1, A_2, \dots, A_n$.
- Substituir, para A_i , produções que começam por A_j , com $j < i$, pelas alternativas de A_j .
- Eliminar a recursão à esquerda imediata de A_i .
- Repetir até que todos os não-terminais tenham sido tratados.

A transformação deve ser validada com cuidado: a gramática resultante deve gerar as mesmas cadeias relevantes, conservar a precedência desejada e não introduzir ambiguidades novas.

Cuidados para a MINIC

- Separar níveis de precedência — expressões lógicas, relacionais, aditivas e multiplicativas.
- Manter a associatividade, especialmente em operadores binários.
- Distinguir chamadas de função, indexação e agrupamento.
- Testar cadeias mínimas, expressões encadeadas e combinações de operadores.

Atividade rápida

Para a gramática $E \rightarrow E + T \mid T$, explique por que um procedimento recursivo para E pode não consumir entrada. Em seguida, aplique a transformação e indique qual produção é escolhida quando o lookahead é `id`.

5. Fatoração à esquerda

A fatoração à esquerda reescreve alternativas que possuem um prefixo comum. O objetivo é permitir que o parser leia o prefixo e adie a escolha até possuir informação suficiente.

Regra geral

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \quad \blacksquare \quad A \rightarrow \alpha A'$$
$$A' \rightarrow \beta_1 \mid \beta_2$$

Exemplo: comandos da MINIC

```
Comando  $\rightarrow$  if ( Expr ) Comando else Comando  
| if ( Expr ) Comando  
| while ( Expr ) Comando  
| id = Expr ;
```

As duas alternativas iniciadas por if compartilham um prefixo. Uma versão fatorada pode ser descrita como:

```
Comando  $\rightarrow$  if ( Expr ) Comando Comando' | while ( Expr ) Comando | id = Expr ;  
Comando'  $\rightarrow$  else Comando |  $\epsilon$ 
```

A fatoração não resolve, sozinha, todos os conflitos — por exemplo, o problema clássico do else associado ao if mais próximo exige uma convenção ou uma gramática cuidadosamente definida. Ela apenas torna explícita a decisão tardia.

6. Conjuntos FIRST

$\text{FIRST}(\alpha)$ é o conjunto de terminais que podem aparecer como primeiro símbolo de alguma cadeia derivada de α . Se α puder derivar ϵ , então ϵ também pertence a $\text{FIRST}(\alpha)$.

Regras essenciais

- Se X é terminal, $\text{FIRST}(X) = \{X\}$.
- Se $X \rightarrow \epsilon$, então $\epsilon \in \text{FIRST}(X)$.
- Para $X \rightarrow Y_1Y_2\dots Y_k$, incluir $\text{FIRST}(Y_1)$ sem ϵ ; se Y_1 for anulável, considerar também Y_2 , e assim por diante.
- Incluir ϵ somente quando todos os símbolos da sequência puderem derivar ϵ .

Exemplo de expressão

Para a gramática fatorada:

$E \rightarrow TE'$; $T \rightarrow FT'$; $F \rightarrow (E) \mid \text{id}$

$\text{FIRST}(F) = \text{FIRST}(T) = \text{FIRST}(E) = \{ (, \text{id} \}$.

Como E' e T' podem produzir ϵ , seus conjuntos FIRST incluem o operador correspondente e ϵ .

Interpretação operacional

Se o lookahead pertence a $\text{FIRST}(\alpha)$, uma produção $A \rightarrow \alpha$ pode ser escolhida sem retrocesso. Por isso, FIRST transforma uma escolha gramatical em uma decisão baseada no próximo token.

7. Conjuntos FOLLOW

FOLLOW(A) é o conjunto de terminais que podem aparecer imediatamente à direita de A em alguma forma sentencial. O marcador \$ representa o fim da entrada.

Regras essenciais

- Colocar \$ em FOLLOW(S), para S símbolo inicial.
- Se houver $A \rightarrow \alpha B \beta$, inserir $\text{FIRST}(\beta) - \{\epsilon\}$ em FOLLOW(B).
- Se $\beta \Rightarrow^* \epsilon$, ou se a produção for $A \rightarrow \alpha B$, inserir FOLLOW(A) em FOLLOW(B).
- Repetir até que nenhum conjunto seja alterado.

Exemplo

Na gramática $E \rightarrow TE'$, como E' aparece no fim da produção, FOLLOW(E) é propagado para FOLLOW(E'). Em $F \rightarrow (E)$, o terminal) pertence a FOLLOW(E). O marcador \$ pertence a FOLLOW(E) quando E é o símbolo inicial.

Uso de FOLLOW

FOLLOW é decisivo para escolher produções ϵ : se $A \rightarrow \epsilon$, essa produção pode ser usada quando o lookahead pertence a FOLLOW(A). O mesmo conjunto também ajuda na recuperação de erros pelo modo pânico, definindo tokens de sincronização.

Figura de apoio do livro



Figura 4.15 — relação conceitual entre FIRST e FOLLOW. Fonte: Aho, Sethi e Ullman, Cap. 4, p. 165.

8. Aplicação ao parser da MINIC

A gramática da MINIC deve ser preparada para que o parser possa decidir a expansão com poucos tokens de lookahead. O processo recomendado é:

- Listar produções para programas, declarações, funções, blocos, comandos e expressões.
- Separar não-terminais por níveis de precedência e associatividade.
- Eliminar recursão à esquerda direta e indireta.
- Fatorar prefixos comuns, sobretudo em comandos iniciados por palavras reservadas.
- Calcular FIRST e FOLLOW para cada não-terminal.
- Registrar conflitos e decidir se a gramática será ajustada ou se será necessário outro método de parsing.

Exemplo de decisão

Se Comando possui alternativas iniciadas por if, while e id, os tokens de lookahead if, while e id orientam escolhas diferentes. Se duas alternativas ainda tiverem o mesmo FIRST, a gramática não está pronta para um parser determinístico de um token.

Integração com a AST

Mesmo em uma abordagem descendente, o parser pode construir a AST durante o reconhecimento: ao reduzir uma expressão fatorada, cria nós para operadores e operandos; ao reconhecer um bloco, agrupa declarações e comandos; ao encontrar uma chamada, registra o identificador e a lista de argumentos.

9. Atividade prática — Etapa 2

Implemente ou documente uma primeira versão do parser descendente da MINIC.

Parte A — Gramática

- Escolha cinco não-terminais centrais e escreva suas produções.
- Identifique uma ocorrência de recursão à esquerda e elimine-a.
- Identifique um prefixo comum e aplique fatoração à esquerda.

Parte B — FIRST e FOLLOW

- Calcule FIRST para os não-terminais escolhidos.
- Calcule FOLLOW, iniciando com \$ no símbolo inicial.
- Justifique cada inclusão de ϵ , terminal ou marcador de fim.

Parte C — Testes

- Use um programa MINIC mínimo com declaração e atribuição.
- Teste uma expressão com precedência, como $a + b * c$.
- Teste blocos aninhados, chamada de função e comando inválido.
- Registre o token de lookahead, a produção escolhida e o resultado.

Entregáveis

Gramática revisada; tabela de FIRST/FOLLOW; descrição das transformações; conjunto de testes; mensagens de erro esperadas; e uma breve reflexão sobre como a gramática influencia a implementação do parser.

10. Síntese e conexão com a Aula 10

A análise top-down começa no símbolo inicial e tenta explicar a entrada por derivações mais à esquerda. Para que a descida recursiva seja previsível, a gramática precisa evitar recursão à esquerda e, quando necessário, ser fatorada.

- Recursão à esquerda pode causar loop infinito.
- Fatoração à esquerda adia decisões com prefixos comuns.
- FIRST indica quais tokens iniciam derivações.
- FOLLOW indica o que pode aparecer após um não-terminal.
- FIRST e FOLLOW sustentam a construção de analisadores LL(1).

Na próxima aula, esses conjuntos serão usados para construir tabelas preditivas, tratar conflitos, implementar o parser não-recursivo e definir estratégias de sincronização para recuperação de erros.

Referências

AHO, A. V.; SETHI, R.; ULLMAN, J. D. Compiladores: princípios, técnicas e ferramentas. Capítulo 4, especialmente as Seções 4.3 e 4.4. Material de referência fornecido para a disciplina. Plano de Ensino de Compiladores — 2026/2.